

1. Comparatie de cod

1.1. Metoda de save pt Bearing:

Lab 1:

```
@Override  Lupse
public Bearing save(Bearing entity) {
    if (entity.getId() == null) {
        entity.setId(UUID.randomUUID());
    }

    if (findById(entity.getId()).isPresent()) {
        return update(entity);
    }
    Connection con = JdbcUtils.getConnection();
    String sql = "INSERT INTO Bearings (bid, diameter, mid) VALUES (?, ?, ?)";
    try {
        PreparedStatement stmt = con.prepareStatement(sql) {

            stmt.setString(1, entity.getId().toString());
            stmt.setLong(2, entity.getDiameter());

            if (entity.getMid() != null) {
                stmt.setLong(3, entity.getMid());
            } else {
                stmt.setNull(3, java.sql.Types.BIGINT);
            }

            stmt.executeUpdate();
            return entity;
        } catch (SQLException e) {
            throw new RuntimeException("Error saving Bearing", e);
        }
    }
}
```

```
private Bearing update(Bearing entity) { 1 usage  Lupse
    String sql = "UPDATE Bearings SET diameter = ?, mid = ? WHERE bid = ?";
    Connection con = JdbcUtils.getConnection();
    try {
        PreparedStatement stmt = con.prepareStatement(sql) {

            stmt.setLong(1, entity.getDiameter());
            if (entity.getMid() != null) {
                stmt.setLong(2, entity.getMid());
            } else {
                stmt.setNull(2, java.sql.Types.BIGINT);
            }
            stmt.setString(3, entity.getId().toString());

            stmt.executeUpdate();
            return entity;
        } catch (SQLException e) {
            throw new RuntimeException("Error updating Bearing", e);
        }
    }
}
```

Lab 3:

```
@Override new *
public Bearing save(Bearing entity) {
    logger.info( message: "Saving bearing with id {}", entity.getId());
    try(var em = entityManagerFactory.createEntityManager()) {
        em.getTransaction().begin();
        try {
            if (entity.getId() == null) {
                logger.info( message: "Creating new bearing {}", entity);
                em.persist(entity);
            } else {
                logger.info( message: "Updating existing bearing {}", entity);
                entity = em.merge(entity);
            }
            em.getTransaction().commit();
            logger.info( message: "Bearing {} committed to database", entity);
            return entity;
        } catch (Exception e) {
            logger.error( message: "Error saving bearing {}: {}", entity, e.getMessage());
            if (em.getTransaction().isActive()) {
                em.getTransaction().rollback();
                logger.warn( message: "Transaction rolled back for bearing {}", entity);
            }
            throw e;
        }
    }
}
```

1.2. Metoda de findById si findAll pt Bearing

Lab1:

```
@Override 7 usages Lupse
public Optional<Bearing> findById(UUID uuid) {
    String sql = "SELECT * FROM Bearings WHERE bid = ?";
    Connection con = JdbcUtils.getConnection();
    try (
        PreparedStatement stmt = con.prepareStatement(sql) {

        stmt.setString( parameterIndex: 1, uuid.toString());
        try (ResultSet rs = stmt.executeQuery()) {
            if (rs.next()) {
                return Optional.of(extractBearing(rs));
            }
        }
    } catch (SQLException e) {
        throw new RuntimeException("Error finding Bearing by ID", e);
    }
    return Optional.empty();
}
```

```

@Override  ⓘ Lups
public List<Bearing> findAll() {
    String sql = "SELECT * FROM Bearings";
    List<Bearing> bearings = new ArrayList<>();
    Connection con = JdbcUtils.getConnection();
    try {
        PreparedStatement stmt = con.prepareStatement(sql);
        ResultSet rs = stmt.executeQuery() {

            while (rs.next()) {
                bearings.add(extractBearing(rs));
            }
            return bearings;
        } catch (SQLException e) {
            throw new RuntimeException("Error fetching all Bearings", e);
        }
    }
}

```

```

private Bearing extractBearing(ResultSet rs) throws SQLException { 2 usages ⓘ Lups
    UUID bid = UUID.fromString(rs.getString(columnLabel: "bid"));
    Long diameter = rs.getLong(columnLabel: "diameter");

    long midVal = rs.getLong(columnLabel: "mid");
    Long mid = rs.isNull() ? null : midVal;

    Bearing bearing = new Bearing();
    bearing.setId(bid);
    bearing.setDiameter(diameter);
    bearing.setMid(mid);
    return bearing;
}

```

Lab3:

```

@Override 7 usages new *
public Optional<Bearing> findById(UUID uuid) {
    logger.info(message: "Finding bearing with id {}", uuid);
    try (var em = entityManagerFactory.createEntityManager()) {
        Bearing bearing = em.find(Bearing.class, uuid);
        if (bearing != null) {
            logger.info(message: "Bearing found: {}", bearing);
        } else {
            logger.info(message: "No bearing found with id {}", uuid);
        }
        return Optional.ofNullable(bearing);
    } catch (Exception e) {
        logger.error(message: "Error finding bearing with id {}: {}", uuid, e.getMessage());
        return Optional.empty();
    }
}

@Override new *
public List<Bearing> findAll() {
    logger.info("Finding all bearings");
    try (var em = entityManagerFactory.createEntityManager()) {
        List<Bearing> bearings = em.createQuery(qlString: "SELECT b FROM Bearing b", Bearing.class).getResultList();
        logger.info(message: "Found {} bearings", bearings.size());
        return bearings;
    } catch (Exception e) {
        logger.error(message: "Error finding all bearings: {}", e.getMessage());
        return List.of();
    }
}

```

1.3. Metoda de delete pt Bearing

Lab1:

```

@Override Lupse
public void deleteById(UUID uuid) {
    String sql = "DELETE FROM Bearings WHERE bid = ?";
    Connection con = JdbcUtils.getConnection();
    try (
        PreparedStatement stmt = con.prepareStatement(sql)) {

        stmt.setString(parameterIndex: 1, uuid.toString());
        stmt.executeUpdate();
    } catch (SQLException e) {
        throw new RuntimeException("Error deleting Bearing", e);
    }
}

```

Lab3:

```

@Override new *
public void deleteById(UUID uuid) {
    logger.info( message: "Deleting bearing with id {}", uuid);
    try (var em = entityManagerFactory.createEntityManager()) {
        em.getTransaction().begin();
        try {
            Bearing bearing = em.find(Bearing.class, uuid);
            if (bearing != null) {
                em.remove(bearing);
                logger.info( message: "Bearing with id {} deleted", uuid);
            } else {
                logger.warn( message: "No bearing found with id {}, nothing to delete", uuid);
            }
            em.getTransaction().commit();
        } catch (Exception e) {
            logger.error( message: "Error deleting bearing with id {}: {}", uuid, e.getMessage());
            if (em.getTransaction().isActive()) {
                em.getTransaction().rollback();
                logger.warn( message: "Transaction rolled back for deleting bearing with id {}", uuid);
            }
            throw e;
        }
    }
}

```

2.1 Metode pentru PalletTruck

Lab1:

```

@Override @Lupse
public PalletTruck save(PalletTruck entity) {
    if (entity.getId() == null) {
        entity.setId(UUID.randomUUID());
    }

    if (findById(entity.getId()).isPresent()) {
        return update(entity);
    }

    String sql = "INSERT INTO PalletTrucks (truck_id, serial_number, type, model, capacity_kg, status, wheels_id) " +
        "VALUES (?, ?, ?, ?, ?, ?, ?)";
    Connection con = JdbcUtils.getConnection();
    try {
        PreparedStatement stmt = con.prepareStatement(sql) {

            stmt.setString( parameterIndex: 1, entity.getId().toString());
            stmt.setString( parameterIndex: 2, entity.getSerialNumber());
            stmt.setString( parameterIndex: 3, entity.getType());
            stmt.setString( parameterIndex: 4, entity.getModel());
            stmt.setLong( parameterIndex: 5, entity.getCapacityKg());
            stmt.setString( parameterIndex: 6, entity.getStatus());
            stmt.setString( parameterIndex: 7, entity.getWheel().getId().toString());

            stmt.executeUpdate();
            return entity;
        } catch (SQLException e) {
            throw new RepositoryException("Error saving PalletTruck", e);
        }
    }
}

```

SAVE

```

private PalletTruck update(PalletTruck entity) {
    String sql = "UPDATE PalletTrucks SET serial_number = ?, type = ?, model = ?, capacity_kg = ?, status = ?, wheels_id = ?" +
        "WHERE truck_id = ?";
    Connection con = JdbcUtils.getConnection();
    try {
        PreparedStatement stmt = con.prepareStatement(sql) {

            stmt.setString(1, entity.getSerialNumber());
            stmt.setString(2, entity.getType());
            stmt.setString(3, entity.getModel());
            stmt.setLong(4, entity.getCapacityKg());
            stmt.setString(5, entity.getStatus());
            stmt.setString(6, entity.getWheel().getId().toString());
            stmt.setString(7, entity.getId().toString());

            stmt.executeUpdate();
            return entity;
        } catch (SQLException e) {
            throw new RepositoryException("Error updating PalletTruck", e);
        }
    }
}

```

UPDATE

```

@Override
public Optional<PalletTruck> findById(UUID uuid) {
    String sql = "SELECT * FROM PalletTrucks WHERE truck_id = ?";
    Connection con = JdbcUtils.getConnection();
    try {
        PreparedStatement stmt = con.prepareStatement(sql) {

            stmt.setString(1, uuid.toString());
            try (ResultSet rs = stmt.executeQuery()) {
                if (rs.next()) {
                    return Optional.of(extractPalletTruck(rs));
                }
            }
        } catch (SQLException e) {
            throw new RepositoryException("Error finding PalletTruck by ID", e);
        }
        return Optional.empty();
    }
}

```

FIND BY ID

```

@Override
public List<PalletTruck> findAll() {
    String sql = "SELECT * FROM PalletTrucks";
    List<PalletTruck> trucks = new ArrayList<>();
    Connection con = JdbcUtils.getConnection();
    try {
        PreparedStatement stmt = con.prepareStatement(sql);
        ResultSet rs = stmt.executeQuery() {

            while (rs.next()) {
                trucks.add(extractPalletTruck(rs));
            }
            return trucks;
        } catch (SQLException e) {
            throw new RepositoryException("Error fetching all PalletTrucks", e);
        }
    }
}

```

FIND ALL

```

@Override  & Lupse
public void deleteById(UUID uuid) {
    String sql = "DELETE FROM PalletTrucks WHERE truck_id = ?";
    Connection con = JdbcUtils.getConnection();
    try {
        PreparedStatement stmt = con.prepareStatement(sql) {
            stmt.setString(1, uuid.toString());
            stmt.executeUpdate();
        } catch (SQLException e) {
            throw new RepositoryException("Error deleting PalletTruck", e);
        }
    }
}

private PalletTruck extractPalletTruck(ResultSet rs) throws SQLException { 2 usages  & Lupse
    PalletTruck truck = new PalletTruck();
    truck.setId(UUID.fromString(rs.getString(columnLabel: "truck_id")));
    truck.setSerialNumber(rs.getString(columnLabel: "serial_number"));
    truck.setType(rs.getString(columnLabel: "type"));
    truck.setModel(rs.getString(columnLabel: "model"));
    truck.setCapacityKg(rs.getLong(columnLabel: "capacity_kg"));
    truck.setStatus(rs.getString(columnLabel: "status"));
    truck.setWheel(getWheel(UUID.fromString(rs.getString(columnLabel: "wheels_id"))));
    return truck;
}

```

```

private Wheel getWheel(UUID wheelId){ 1 usage  & Lupse

    var wheelOp = wheelRepo.findById(wheelId);
    if(wheelOp.isPresent()){
        return wheelOp.get();
    }
    throw new RepositoryException("Wheel with ID " + wheelId + " not found");
}

```

LAB 3:

```

@Override new *
public PalletTruck save(PalletTruck entity) {
    logger.info(message: "Saving pallet truck {}", entity);

    try (EntityManager em = entityManagerFactory.createEntityManager()) {
        em.getTransaction().begin();

        try {
            if (entity.getId() == null) {
                logger.info(message: "Creating pallet truck {}", entity);
                em.persist(entity);
            } else {
                logger.info(message: "Updating pallet truck {}", entity);
                entity = em.merge(entity);
            }

            em.getTransaction().commit();
            logger.info(message: "Pallet truck {} committed to database", entity);
            return entity;
        } catch (Exception e) {
            logger.error(message: "Error saving pallet truck {}: {}", entity, e.getMessage());
            if (em.getTransaction().isActive()) {
                em.getTransaction().rollback();
                logger.warn(message: "Transaction rolled back for pallet truck {}", entity);
            }
            throw e;
        }
    }
}

```

SAVE + UPDATE

```

@Override 7 usages new *
public Optional<PalletTruck> findById(UUID uuid) {
    logger.info(message: "Finding pallet truck with id {}", uuid);

    try (EntityManager em = entityManagerFactory.createEntityManager()) {
        PalletTruck truck = em.find(PalletTruck.class, uuid);
        if (truck != null) {
            logger.info(message: "Pallet truck found: {}", truck);
            return Optional.of(truck);
        } else {
            logger.warn(message: "Pallet truck with id {} not found", uuid);
            return Optional.empty();
        }
    }
}

```

FIND BY ID

```

@Override new *
public List<PalletTruck> findAll() {
    logger.info("Finding all pallet trucks");

    try (EntityManager em = entityManagerFactory.createEntityManager()) {
        return em.createQuery(qlString: "SELECT p FROM PalletTruck p", PalletTruck.class).getResultList()
    }
}

```

FIND ALL

```

@Override new *
public void deleteById(UUID uuid) {
    logger.info(message: "Deleting pallet truck with id {}", uuid);

    try (EntityManager em = entityManagerFactory.createEntityManager()) {
        em.getTransaction().begin();

        try {
            PalletTruck truck = em.find(PalletTruck.class, uuid);
            if (truck != null) {
                em.remove(truck);
                em.getTransaction().commit();
                logger.info(message: "Pallet truck with id {} deleted", uuid);
            } else {
                logger.warn(message: "Pallet truck with id {} not found for deletion", uuid);
                em.getTransaction().rollback();
            }
        } catch (Exception e) {
            logger.error(message: "Error deleting pallet truck with id {}: {}", uuid, e.getMessage());
            if (em.getTransaction().isActive()) {
                em.getTransaction().rollback();
            }
            throw e;
        }
    }
}
}

```

DELETE

3. Metode pentru Wheel

Lab1:

```

@Override
public Wheel save(Wheel entity) {
    if (entity.getId() == null) {
        entity.setId(UUID.randomUUID());
    }

    if (findById(entity.getId()).isPresent()) {
        return update(entity);
    }

    String sql = "INSERT INTO Wheels (wheels_id, materials_id, max_weight, bid) VALUES (?, ?, ?, ?)";
    Connection con = JdbcUtils.getConnection();
    try {
        PreparedStatement stmt = con.prepareStatement(sql) {

            stmt.setString(1, entity.getId().toString());
            stmt.setString(2, entity.getMaterial().getId().toString());
            stmt.setLong(3, entity.getMaxWeight());
            stmt.setString(4, entity.getBearing().getId().toString());

            stmt.executeUpdate();
            return entity;
        } catch (SQLException e) {
            throw new RuntimeException("Error saving Wheel", e);
        }
    }
}

```

SAVE

```

private Wheel update(Wheel entity) {
    String sql = "UPDATE Wheels SET materials_id = ?, max_weight = ?, bid = ? WHERE wheels_id = ?";
    Connection con = JdbcUtils.getConnection();
    try {
        PreparedStatement stmt = con.prepareStatement(sql) {

            stmt.setString(1, entity.getMaterial().getId().toString());
            stmt.setLong(2, entity.getMaxWeight());
            stmt.setString(3, entity.getBearing().getId().toString());
            stmt.setString(4, entity.getId().toString());

            stmt.executeUpdate();
            return entity;
        } catch (SQLException e) {
            throw new RuntimeException("Error updating Wheel", e);
        }
    }
}

```

UPDATE

```

@Override 7 usages  Lupse
public Optional<Wheel> findById(UUID uuid) {
    String sql = "SELECT * FROM Wheels WHERE wheels_id = ?";
    Connection con = JdbcUtils.getConnection();
    try (
        PreparedStatement stmt = con.prepareStatement(sql)) {

        stmt.setString(1, uuid.toString());
        try (ResultSet rs = stmt.executeQuery()) {
            if (rs.next()) {
                return Optional.of(extractWheel(rs));
            }
        }
    } catch (SQLException e) {
        throw new RuntimeException("Error finding Wheel by ID", e);
    }
    return Optional.empty();
}

```

FIND BY ID

```

@Override  Lupse
public List<Wheel> findAll() {
    String sql = "SELECT * FROM Wheels";
    List<Wheel> wheels = new ArrayList<>();
    Connection con = JdbcUtils.getConnection();
    try (
        PreparedStatement stmt = con.prepareStatement(sql);
        ResultSet rs = stmt.executeQuery()) {

        while (rs.next()) {
            wheels.add(extractWheel(rs));
        }
        return wheels;
    } catch (SQLException e) {
        throw new RuntimeException("Error fetching all Wheels", e);
    }
}

```

FIND ALL

```
@Override  👤 Lupsé
public void deleteById(UUID uuid) {
    String sql = "DELETE FROM Wheels WHERE wheels_id = ?";
    Connection con = JdbcUtils.getConnection();
    try (
        PreparedStatement stmt = con.prepareStatement(sql)) {

        stmt.setString( parameterIndex: 1, uuid.toString());
        stmt.executeUpdate();
    } catch (SQLException e) {
        throw new RuntimeException("Error deleting Wheel", e);
    }
}
```

DELETE

```

private Wheel extractWheel(ResultSet rs) throws SQLException { 2 usages  ⚡ Lapse
    Wheel wheel = new Wheel();
    wheel.setId(UUID.fromString(rs.getString(columnLabel: "wheels_id")));
    wheel.setMaterial(getWheelMaterial(UUID.fromString(rs.getString(columnLabel: "materials_id"))));
    wheel.setMaxWeight(rs.getLong(columnLabel: "max_weight"));
    wheel.setBearing(getBearing(UUID.fromString(rs.getString(columnLabel: "bid"))));
    return wheel;
}

private Bearing getBearing(UUID id){ 1 usage  ⚡ Lapse
    String sql = "SELECT * FROM Bearings WHERE bid = ?";
    Bearing bearing = new Bearing();
    Connection con = JdbcUtils.getConnection();
    try (
        PreparedStatement stmt = con.prepareStatement(sql)) {

        stmt.setString(parameterIndex: 1, id.toString());
        try (ResultSet rs = stmt.executeQuery()) {
            if (rs.next()) {
                bearing.setId(UUID.fromString(rs.getString(columnLabel: "bid")));
                bearing.setDiameter(rs.getLong(columnLabel: "diameter"));
                bearing.setMid(rs.getLong(columnLabel: "mid"));
            }
        }
    } catch (SQLException e) {
        throw new RuntimeException("Error finding Bearing by ID", e);
    }
    return bearing;
}

```

```

private WheelMaterial getWheelMaterial(UUID id){ 1 usage  ⚡ Lapse
    String sql = "SELECT * FROM WheelMaterials WHERE materials_id = ?";
    WheelMaterial wheelMaterial = new WheelMaterial();
    Connection con = JdbcUtils.getConnection();
    try (
        PreparedStatement stmt = con.prepareStatement(sql)) {

        stmt.setString(parameterIndex: 1, id.toString());
        try (ResultSet rs = stmt.executeQuery()) {
            if (rs.next()) {
                wheelMaterial.setId(UUID.fromString(rs.getString(columnLabel: "materials_id")));
                wheelMaterial.setType(rs.getString(columnLabel: "type"));
                wheelMaterial.setMaxWeight(rs.getLong(columnLabel: "max_weight"));
            }
        }
    } catch (SQLException e) {
        throw new RuntimeException("Error finding WheelMaterial by ID", e);
    }
    return wheelMaterial;
}

```

HELPERS

LAB 3:

```

@Override new *
public Wheel save(Wheel entity) {
    logger.info( message: "Saving wheel {}", entity);

    try (var em = entityManagerFactory.createEntityManager()) {
        em.getTransaction().begin();

        try {
            if (entity.getId() == null) {
                logger.info( message: "Creating wheel {}", entity);
                em.persist(entity);
            } else {
                logger.info( message: "Updating wheel {}", entity);
                entity = em.merge(entity);
            }

            em.getTransaction().commit();
            logger.info( message: "Wheel {} committed to database", entity);
            return entity;

        } catch (Exception e) {
            logger.error( message: "Error saving wheel {}: {}", entity, e.getMessage());
            if (em.getTransaction().isActive()) {
                em.getTransaction().rollback();
                logger.warn( message: "Transaction rolled back for wheel {}", entity);
            }
            throw e;
        }
    }
}

```

SAVE+UPDATE

```

@Override 7 usages new *
public Optional<Wheel> findById(UUID uuid) {
    logger.info( message: "Finding wheel {}", uuid);
    try(var em = entityManagerFactory.createEntityManager()) {
        Wheel wheel = em.find(Wheel.class, uuid);
        if(wheel != null) {
            logger.info( message: "Wheel {} found", wheel);
        } else {
            logger.info( message: "Wheel with id {} not found", uuid);
        }
        return Optional.ofNullable(wheel);
    } catch (Exception e) {
        logger.error( message: "Error finding wheel with id {}: {}", uuid, e.getMessage());
        return Optional.empty();
    }
}

```

FIND BY ID

```

@Override new *
public List<Wheel> findAll() {
    logger.info("Finding all wheels");
    try(var em = entityManagerFactory.createEntityManager()) {
        List<Wheel> wheels = em.createQuery(qlString: "SELECT w FROM Wheel w JOIN fetch w.material JOIN fetch w.bearing", Wheel.class).getResultList();
        logger.info(message: "{} wheels found", wheels.size());
        return wheels;
    } catch (Exception e) {
        logger.error(message: "Error finding all wheels: {}", e.getMessage());
        throw e;
    }
}

```

FIND ALL

```

@Override new *
public void deleteById(UUID uuid) {
    logger.info(message: "Deleting wheel {}", uuid);
    try(var em = entityManagerFactory.createEntityManager()) {
        em.getTransaction().begin();
        Wheel wheel = em.find(Wheel.class, uuid);
        try{
            if(wheel != null) {
                em.remove(wheel);
                logger.info(message: "Wheel {} deleted", wheel);
            } else {
                logger.warn(message: "Wheel with id {} not found for deletion", uuid);
            }
            em.getTransaction().commit();
        } catch (Exception e) {
            logger.error(message: "Error deleting wheel with id {}: {}", uuid, e.getMessage());
            if (em.getTransaction().isActive()) {
                em.getTransaction().rollback();
                logger.warn(message: "Transaction rolled back for wheel deletion with id {}", uuid);
            }
            throw e;
        }
    } catch (Exception e) {
        logger.error(message: "Error deleting wheel with id {}: {}", uuid, e.getMessage());
        throw e;
    }
}

```

DELETE

4. Metode pentru Wheel Material

Lab 1:

```

@Override  Lupse
public WheelMaterial save(WheelMaterial entity) {
    if (entity.getId() == null) {
        enti
    }
    if (find
        retu
    }

    String sql = "INSERT INTO WheelMaterials (materials_id, type, max_weight) VALUES (?, ?, ?)";
    Connection con = JdbcUtils.getConnection();

    try (
        PreparedStatement stmt = con.prepareStatement(sql)) {

        stmt.setString(parameterIndex: 1, entity.getId().toString());
        stmt.setString(parameterIndex: 2, entity.getType());
        stmt.setLong(parameterIndex: 3, entity.getMaxWeight());

        stmt.executeUpdate();
        return entity;
    } catch (SQLException e) {
        throw new RuntimeException("Error saving WheelMaterial", e);
    }
}

```

SAVE

```

private WheelMaterial update(WheelMaterial entity) { 1 usage  Lupse
    String sql = "UPDATE WheelMaterials SET type = ?, max_weight = ? WHERE materials_id = ?";
    Connection con = JdbcUtils.getConnection();

    try (
        PreparedStatement stmt = con.prepareStatement(sql)) {

        stmt.setString(parameterIndex: 1, entity.getType());
        stmt.setLong(parameterIndex: 2, entity.getMaxWeight());
        stmt.setString(parameterIndex: 3, entity.getId().toString());

        stmt.executeUpdate();
        return entity;
    } catch (SQLException e) {
        throw new RuntimeException("Error updating WheelMaterial", e);
    }
}

```

UPDATE

```

@Override 7 usages  Lupse
public Optional<WheelMaterial> findById(UUID uuid) {
    String sql = "SELECT * FROM WheelMaterials WHERE materials_id = ?";
    Connection con = JdbcUtils.getConnection();

    try (
        PreparedStatement stmt = con.prepareStatement(sql)) {

        stmt.setString(1, uuid.toString());
        try (ResultSet rs = stmt.executeQuery()) {
            if (rs.next()) {
                return Optional.of(extractWheelMaterial(rs));
            }
        }
    } catch (SQLException e) {
        throw new RuntimeException("Error finding WheelMaterial by ID", e);
    }
    return Optional.empty();
}

```

FIND BY ID

```

@Override  Lupse
public List<WheelMaterial> findAll() {
    String sql = "SELECT * FROM WheelMaterials";
    List<WheelMaterial> materials = new ArrayList<>();
    Connection con = JdbcUtils.getConnection();

    try (
        PreparedStatement stmt = con.prepareStatement(sql);
        ResultSet rs = stmt.executeQuery()) {

        while (rs.next()) {
            materials.add(extractWheelMaterial(rs));
        }
        return materials;
    } catch (SQLException e) {
        throw new RuntimeException("Error fetching all WheelMaterials", e);
    }
}

```

FIND ALL

```

@Override  Lups
public void deleteById(UUID uuid) {
    String sql = "DELETE FROM WheelMaterials WHERE materials_id = ?";
    Connection con = JdbcUtils.getConnection();

    try (
        PreparedStatement stmt = con.prepareStatement(sql)) {

        stmt.setString(1, uuid.toString());
        stmt.executeUpdate();
    } catch (SQLException e) {
        throw new RuntimeException("Error deleting WheelMaterial", e);
    }
}

```

DELETE

```

private WheelMaterial extractWheelMaterial(ResultSet rs) throws SQLException { 2 usages  Lups
    WheelMaterial material = new WheelMaterial();
    material.setId(UUID.fromString(rs.getString(columnLabel: "materials_id")));
    material.setType(rs.getString(columnLabel: "type"));
    material.setMaxWeight(rs.getLong(columnLabel: "max_weight"));
    return material;
}

```

HELPER

LAB 3:

```

@Override new *
public WheelMaterial save(WheelMaterial entity) {
    logger.info( message: "Saving wheel material {}", entity);

    try (var em = entityManagerFactory.createEntityManager()) {
        em.getTransaction().begin();

        try {
            if (entity.getId() == null) {
                logger.info( message: "Creating wheel material {}", entity);
                em.persist(entity);
            } else {
                logger.info( message: "Updating wheel material {}", entity);
                entity = em.merge(entity);
            }

            em.getTransaction().commit();
            logger.info( message: "Wheel material {} committed to database", entity);
            return entity;

        } catch (Exception e) {
            logger.error( message: "Error saving wheel material {}: {}", entity, e.getMessage());
            if (em.getTransaction().isActive()) {
                em.getTransaction().rollback();
                logger.warn( message: "Transaction rolled back for wheel material {}", entity);
            }
            throw e;
        }
    }
}

```

SAVE + UPDATE

```

@Override 7 usages new *
public Optional<WheelMaterial> findById(UUID uuid) {
    logger.info( message: "Finding wheel material by id {}", uuid);
    try (var em = entityManagerFactory.createEntityManager()) {
        WheelMaterial material = em.find(WheelMaterial.class, uuid);
        if (material != null) {
            logger.info( message: "Wheel material found: {}", material);
            return Optional.of(material);
        } else {
            logger.info( message: "No wheel material found with id {}", uuid);
            return Optional.empty();
        }
    } catch (Exception e) {
        logger.error( message: "Error finding wheel material with id {}: {}", uuid, e.getMessage());
        throw e;
    }
}

```

FIND BY ID

```

@Override new *
public List<WheelMaterial> findAll() {
    logger.info("Finding all wheel material entities");
    try (var em = entityManagerFactory.createEntityManager()) {
        var query = em.createQuery(qlString: "SELECT wm FROM WheelMaterial wm", WheelMaterial.class);
        List<WheelMaterial> materials = query.getResultList();
        logger.info(message: "Found {} wheel material entities", materials.size());
        return materials;
    } catch (Exception e) {
        logger.error(message: "Error finding all wheel material entities: {}", e.getMessage());
        throw e;
    }
}

```

FIND ALL

```

@Override new *
public void deleteById(UUID uuid) {
    logger.info(message: "Deleting wheel material by id {}", uuid);
    try (var em = entityManagerFactory.createEntityManager()) {
        em.getTransaction().begin();
        WheelMaterial material = em.find(WheelMaterial.class, uuid);
        if (material != null) {
            em.remove(material);
            logger.info(message: "Wheel material with id {} deleted", uuid);
        } else {
            logger.warn(message: "No wheel material found with id {}, nothing to delete", uuid);
        }
        em.getTransaction().commit();
    } catch (Exception e) {
        logger.error(message: "Error deleting wheel material with id {}: {}", uuid, e.getMessage());
        throw e;
    }
}

```

DELETE

Hibernate ajuta la reducerea codului boilerplate deoarece nu mai am nevoie de functii helper separate pentru a mapa obiecte si pt a obtine obiecte tinute de alte obiecte prin compozitie, si ajuta la centralizare de save si update intr-un singur loc mai usor.

Mentinerea schemei de baze de date e mai usor de intretinut up-to-date fara cunostinte sql doar prin adaugarea adnotarilor si ofera manipulare mai usoara a bazei de date prin cod dor prin adaugarea proprietatii si capacitatea de a seta show_sql la true ne ajuta la debugging:

```

<property name="hibernate.show_sql" value="true"/>
<property name="hibernate.hbm2ddl.auto" value="update"/>

```

Implementare connection pooling

```
private static synchronized void initPool() { 1 usage new *
    if (dataSource != null) return;
    if(Objects.isNull(jdbcProps)){
        initAppConfig();
    }

    HikariConfig config = new HikariConfig();

    config.setJdbcUrl(jdbcProps.getProperty("jdbc.url"));
    config.setUsername(jdbcProps.getProperty("jdbc.username"));
    config.setPassword(jdbcProps.getProperty("jdbc.password"));

    config.setMaximumPoolSize(100);
    config.setMinimumIdle(2);
    config.setConnectionTimeout(3000);
    config.setIdleTimeout(600000);
    config.setMaxLifetime(1800000);

    dataSource = new HikariDataSource(config);
    System.out.println("HikariCP Connection Pool initialized!");
}
```

Este mai avantajos sa folosim connection pooling deoarece timpul de obtinere a unei conexiuni este mai scurt deoarece un pool contine un anumit numar de conexiune deja create iar prin apelarea metodei getConnection obtinem o conexiune mai rapid decat daca am creuna una de fiecare data cand apelam metoda

Testare

1. Crearea a 100 de conexiuni la baza de date fără pooling (creați conexiune nouă de fiecare dată)

```

private static void testNoPooling(int testSize){ 1 usage new *
    Timestamp now = new Timestamp(System.currentTimeMillis());
    List<Double> connTimes = new ArrayList<>();
    for (int i = 0; i < testSize; i++) {
        long startConn = System.nanoTime();
        try (
            Connection connection = DriverManager.getConnection(
                jdbcProps.getProperty("jdbc.url"),
                jdbcProps.getProperty("jdbc.username"),
                jdbcProps.getProperty("jdbc.password"))){
            long endConn = System.nanoTime();
            var stmt = connection.createStatement();
            stmt.execute( sql: "SELECT 1");

            connTimes.add((endConn - startConn) / 1_000_000.0);
        } catch (SQLException e) {
            throw new RuntimeException("CRITICAL: Failed to get database connection!", e);
        }
    }

    double averageConnTime = connTimes.stream().mapToDouble(Double::doubleValue).average().orElse( other: 0.0);
    System.out.println("Average connection time without pooling: " + averageConnTime + " ms");
    Timestamp after = new Timestamp(System.currentTimeMillis());
    System.out.println("Time taken without pooling: " + (after.getTime() - now.getTime()) + " ms");
}

```

2. Obținerea a 100 de conexiuni cu pooling (din pool-ul de conexiuni)

```

private static void testPooling(int testSize){ 1 usage new *
    Timestamp now = new Timestamp(System.currentTimeMillis());
    List<Double> connTimes = new ArrayList<>();

    for (int i = 0; i < testSize; i++) {
        long startConn = System.nanoTime();
        try (
            Connection connection = JdbcHikariUtils.getConnection();) {

            long endConn = System.nanoTime();
            var stmt = connection.createStatement();
            stmt.execute( sql: "SELECT 1");

            connTimes.add((endConn - startConn) / 1_000_000.0);
        } catch (SQLException e) {
            throw new RuntimeException("CRITICAL: Failed to get database connection!", e);
        }
    }

    double averageConnTime = connTimes.stream().mapToDouble(Double::doubleValue).average().orElse( other: 0.0);
    System.out.println("Average connection time with pooling: " + averageConnTime + " ms");
    Timestamp after = new Timestamp(System.currentTimeMillis());
    System.out.println("Time taken with pooling: " + (after.getTime() - now.getTime()) + " ms");
}

```

Rezultate rulare ambele cu parametri 100 pentru testsize

```
> Task :compileJava
> Task :processResources UP-TO-DATE
> Task :classes

> Task :org.example.lab3.PerformanceTest.main()
Properties loaded successfully!
Average connection time without pooling: 124.30352199999999 ms
Time taken without pooling: 12545 ms
Properties loaded successfully!
log4j:WARN No appenders could be found for logger (com.zaxxer.hikari.HikariConfig).
log4j:WARN Please initialize the log4j system properly.
log4j:WARN See http://logging.apache.org/log4j/1.2/faq.html#noconfig for more info.
HikariCP Connection Pool initialized!
Average connection time with pooling: 1.71723 ms
Time taken with pooling: 214 ms
```

În abordarea standard, solicitarea manuală a unei conexiuni a durat în medie 124.3 ms, conducând la un timp total de 12.5 secunde pentru 100 de operațiuni, din cauza overhead-ului repetat generat de rețea, criptare și autentificare. Prin implementarea HikariCP, latența medie a scăzut la doar 1.7 ms, reducând timpul total de execuție la 214 milisecunde. Această îmbunătățire de aproape 60 de ori confirmă că reutilizarea conexiunilor deschise în memorie elimină blocajele de rețea și este o optimizare esențială pentru scalabilitatea aplicației.

Detectarea Scurgerilor de Conexiuni

1. Creați un scenariu în care conexiunile sunt deschise dar nu închise

```
private static void testNoConnectionClose(int testSize){ 1 usage new *

    for (int i = 0; i < testSize; i++) {
        System.out.println("Iteration " + (i) + ": Attempting to get connection...");

        try {
            Connection connection = JdbcHikariUtils.getConnection();

            var stmt = connection.createStatement();
            stmt.execute( sql: "SELECT 1");

        } catch (SQLTransientConnectionException e) {
            System.out.println("Connection pool exhausted after " + i + " iterations.");
            break;
        } catch (SQLException e) {
            throw new RuntimeException("Failed to get database connection!", e);
        } finally {
            System.out.println("Keeping connection " + i);
        }
    }
}
```

2.

Arătați că pool-ul devine epuizat

```
Iteration 90: Attempting to get connection...
Keeping connection 90
Iteration 91: Attempting to get connection...
Keeping connection 91
Iteration 92: Attempting to get connection...
Keeping connection 92
Iteration 93: Attempting to get connection...
Keeping connection 93
Iteration 94: Attempting to get connection...
Keeping connection 94
Iteration 95: Attempting to get connection...
Keeping connection 95
Iteration 96: Attempting to get connection...
Keeping connection 96
Iteration 97: Attempting to get connection...
Keeping connection 97
Iteration 98: Attempting to get connection...
Keeping connection 98
Iteration 99: Attempting to get connection...
Keeping connection 99
Iteration 100: Attempting to get connection...
Connection pool exhausted after 100 iterations.
Keeping connection 100
Iteration 0: Attempting to get connection...
```

Cand connection pool-ul devine epuizata se asteapta ca Connection timeout –ul sa expire apoi se arunca eroarea de `SqlTransientConnectionExpired` ce semnaleaza ca nu mai sunt conexiuni disponibile in pool

3. Remediați problema cu gestionarea corectă a resurselor

```
private static void testNoConnectionCloseFixed(int testSize){ 1 usage new *
    for (int i = 0; i < testSize; i++) {
        System.out.println("Iteration " + (i) + ": Attempting to get connection...");

        try
            (Connection connection = JdbcHikariUtils.getConnection());{

            var stmt = connection.createStatement();
            stmt.execute( sql: "SELECT 1");

        } catch (SQLTransientConnectionException e) {
            System.out.println("Connection pool exhausted after " + i + " iterations.");
            break;
        } catch (SQLException e) {
            throw new RuntimeException("Failed to get database connection!", e);
        } finally {
            System.out.println("Closed connection " + i);
        }
    }
}
```

Peentru remediere trebuie doar sa apelam metoda close(lucur ce se face automat prin try with resources) de la wrapperul conexiuni, care nu inchide conexiunea ci o pune inapoi in pool

Configurați relația părinte-copil să folosească lazy loading implicit

```
@ManyToOne(fetch = FetchType.LAZY) 3 usages
@JoinColumn(name = "wheels_id", nullable = false)
private Wheel wheel;
```

La startul aplicatie un PalletTruck nu are nevoie de roata in sine e de ajuns doar proxy ul pentru ca in tabel apar doar date care nu depind de roata, iar pentru operatiile aplicate de save, update unde e nevoie de roate aceasta trebuie selectata inainte , iar pentru delete nu e nevoie de incarcare roata.

Implementați cel puțin un scenariu unde aveți nevoie de eager loading

```

private UUID id;

@ManyToOne(fetch = FetchType.EAGER) 3 usages
@JoinColumn(name = "materials_id", nullable = false)
private WheelMaterial material;

@ManyToOne(fetch = FetchType.EAGER) 3 usages
@JoinColumn(name = "bid", nullable = false)
private Bearing bearing;

@Column(name = "max_weight", nullable = false) 2 usages

```

In cazul rotii avem nevoie de material si beraing din prima deoarece aceste informatii sunt afisate in tabel si sunt necesare pentru specificarea rotii

Demonstrați diferența în interogările SQL generate

In Hibernate, strategia EAGER incarca automat toate entitatile relationate odata cu cea principala, ceea ce sufoca baza de date provocand problema N+1 interogari. Strategia LAZY este mult mai eficienta deoarece incarca datele doar la cerere, folosind initial un obiect fals (Proxy). Solutia optima este setarea tuturor relatiilor pe LAZY pentru performanta si folosirea explicita a lui JOIN FETCH in interogari pentru a extrage toate datele necesare intr-un singur apel, evitand astfel atat interogari multiple, cat si erorile de conexiune inchisa.

Interogari EAGER cu join fetch pentru a nu avea N + 1

```

Hibernate:
  select
    w1_0.wheels_id,
    w1_0.bid,
    b1_0.bid,
    b1_0.diameter,
    b1_0.mid,
    w1_0.materials_id,
    m1_0.materials_id,
    m1_0.max_weight,
    m1_0.type,
    w1_0.max_weight
  from
    Wheels w1_0
  join
    WheelMaterials m1_0
      on m1_0.materials_id=w1_0.materials_id
  join
    Bearings b1_0
      on b1_0.bid=w1_0.bid

```

Interogare pnetru a aduce toate PalletTrucks

```
Hibernate:
  select
    pt1_0.truck_id,
    pt1_0.capacity_kg,
    pt1_0.model,
    pt1_0.serial_number,
    pt1_0.status,
    pt1_0.type,
    pt1_0.wheels_id
  from
    PalletTrucks pt1_0
```

se Observa ca la pallettruck nu se aduce nici un wheel obiectul fiind inlocuit de un proxy

Avantaje ORM :

Mai usor de folosit, usor de intretinut, ajuta sa te focusezi pe alte lucruri fata de conexiunea cu baze date, mai putin cod, mai usor de dat debug

Dezavantaje:

Dureaza configurarea

Nu am intampinat provocari

Comparatie SQL generat cu Sql normal

Sql:

Urmeaza format standar mai greu de citit in unele cazuri mai putin performante daca nu folosim join fetch rezulta n+1 in unele cazuri

```
Hibernate:
select
    w1_0.wheels_id,
    w1_0.bid,
    b1_0.bid,
    b1_0.diameter,
    b1_0.mid,
    w1_0.materials_id,
    m1_0.materials_id,
    m1_0.max_weight,
    m1_0.type,
    w1_0.max_weight
from
    Wheels w1_0
```

Sql normal:

Mai usor de citit si putem scrie cod sql mai optimizat

SQL-ul nativ ofera o performanta bruta superioara deoarece elimina complet "overhead-ul" introdus de ORM, cum ar fi generarea interogarilor la runtime, maparea rezultatelor pe obiecte complexe si mentinerea starii entitatilor in memorie, permitand totodata utilizarea optimizarilor specifice bazei de date. In schimb, SQL-ul generat de un ORM (precum Hibernate) sacrifica o mica fractiune din aceasta viteza in favoarea productivitatii masive, portabilitatii intre baze de date diferite si securitatii (protectie anti-injectie), diferenta de performanta devenind adesea neglijabila in aplicatiile moderne daca sunt folosite corect strategii precum lazy loading si connection pooling.